

# 函数式接口及其成员

---

北京理工大学计算机学院  
金旭亮



# 函数式接口



能接收一个Lambda表达式的变量，必须是接口类型，并且这种接口，还必须是一种“**函数式接口 (functional interface)**”。



所谓“函数式接口”，就是“只定义有一个**抽象**方法的接口”，在Java 8之前，这种接口被称为“SAM: Single Abstract Method”接口。



Java 8中，使用“**@FunctionalInterface**”标识一个“函数式接口”。

```
@FunctionalInterface
public interface Runnable {
    public abstract void run();
}
```



JDK 8中，Runnable就是一个函数式接口

# 函数描述符

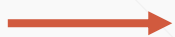


函数式接口就是只定义一个抽象方法的接口。这个抽象方法，它的名字不重要，它的输入参数和返回值类型才重要。



函数式接口的抽象方法的签名（函数签名，functional method signature）称为“**函数描述符**”。在实际开发时，编译器会使用这个信息对Lambda表达式进行推断，以确定此Lambda表达式是否符合此函数式接口的要求。

```
@FunctionalInterface  
public interface Runnable {  
    void run();  
}
```



() -> void

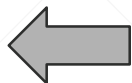
函数签名

# 什么样的接口才算是“函数型接口”？



最主要且唯一的标准：只定义有一个抽象方法。

```
@FunctionalInterface
interface MyInterface{
    void f();
    void g();
}
```



此方法不是函数型接口，无法通过编译，因为它定义了两个抽象方法，并且使用了@FunctionalInterface标识。

在统计某接口抽象方法的“个数”时，以下种类的方法是不计算在内的：

1. 默认方法 (Default methods)
2. 静态方法 (Static methods)
3. Object类所定义的公有方法

Java 8 新增特性，  
后面PPT介绍

# 默认方法 (default method)

默认方法，使用default关键字声明：

```
@FunctionalInterface
interface Printer {
    //唯一的抽象方法
    void print(String msg);
    //默认方法 ( Java 8+)
    default void welcome(){
        print("Hello");
    }
}
```



上述代码中突出显示的部分称为“**默认方法 (Default Method)**”，是Java 8引入的特性。

# 使用默认方法

```
public class UserPrinterInterface {  
    public int value;  
    public static void main(String[] args) {  
        Printer printer = (msg)->System.out.println(msg);  
        printer.welcome();  
    }  
}
```

在接口中定义一个默认方法，实际上意味着：

只要实现了这个接口的类，就自动拥有这个方法，并且这个方法已经写好了，定义新类时无需再手工重写一遍。

# 静态方法

除了默认方法，在接口中还可以定义静态方法：

```
@FunctionalInterface
interface Printer {
    void print(String msg);
    //静态方法 (Java 8+)
    static void staticMethod(){
        System.out.println("static method of Printer");
    }
}
```

定义

```
public class Test {
    public int value;
    public static void main(String[] args) {
        Printer.staticMethod();
    }
}
```

使用

# 关于接口中的静态方法

对于接口中的静态方法，你可以这样理解：

只要实现了这个接口的所有类，都自动拥有这个静态方法。



# 为什么允许在接口中定义默认方法和静态方法？

- 1 减少不必要的类型定义，避免“类型数目爆炸”。
- 2 在技术演进时，向前保持代码的兼容性。

# 出现冲突怎么办？



如果接口中定义了某默认方法（或静态方法），实现此接口的类中也定义了同名的方法，会出现什么情况呢？



请你自己设计实例，看看当这种情况出现时，Java编译器是怎样应对的。

# 函数式接口可包容成员类型汇总

```
@FunctionalInterface
public interface MyFunctionalInterface {
    //接口常量
    final int MAX_VALUE = 255;
    //唯一的抽象公有方法
    void func();
    //放置Object类的公有方法
    int hashCode();
    String toString();
    boolean equals(Object obj);

    private void instancePrivateMethod() {
        System.out.println("JDK9新增: 接口中的私有实例方法");
    }

    default void defaultMethod() {
        System.out.println("JDK8新增: 接口中的默认方法");
        instancePrivateMethod(); //调用接口中的私有实例方法
    }
}
```

定义函数式接口时，`@FunctionalInterface`注解不是必需的，但如果你加入了这个注解，而你定义的接口又不符合函数式接口的要求时，则Java编译器会给出提示，要你修正错误。所以，推荐加入这个注解。

```
private static void staticPrivateMethod() {
    System.out.println("JDK9新增: 接口中的私有静态方法");
}

static void staticMethod() {
    System.out.println("JDK8新增: 接口中的静态方法");
    staticPrivateMethod(); //调用接口中的私有静态方法
}
```

MyFunctionalInterface.java

# 实现函数式接口

```
//此类自动拥有接口所定义的默认方法和静态方法
class MyFunctionalClass implements MyFunctionalInterface {
    @Override
    public void func() {
        System.out.println("调用MyFunctionalClass.func()");
    }
}
```

实现了函数式接口的具体类型，自动拥有接口中所定义的默认方法和静态方法。

# 使用函数型接口示例

运行结果:

```
Run: UseMyFunctionalInterface x
"C:\Program Files\Java\jdk-17\bin\java.exe"
JDK8新增: 接口中的静态方法
JDK9新增: 接口中的私有静态方法
255
=====

调用MyFunctionalClass.func()
JDK8新增: 接口中的默认方法
JDK9新增: 接口中的私有实例方法
990368553

=====

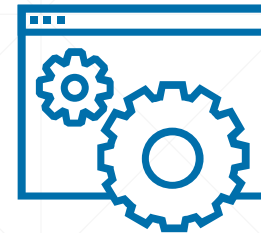
将Lambda表达式赋值给接口变量
JDK8新增: 接口中的默认方法
JDK9新增: 接口中的私有实例方法
1078694789

Process finished with exit code 0
```



```
class UseMyFunctionalInterface {
    public static void main(String[] args) {
        //调用接口的静态方法
        MyFunctionalInterface.staticMethod();
        //访问接口中的常量字段
        System.out.println(MyFunctionalInterface.MAX_VALUE);
        System.out.println("\n=====");
        //实例化一个实现了接口的类
        MyFunctionalInterface obj = new MyFunctionalClass();
        //通过接口变量访问接口中的成员
        obj.func();
        obj.defaultMethod();
        System.out.println(obj.hashCode());
        System.out.println("\n=====");
        //让接口变量接收一个Lambda表达式
        obj = () -> System.out.println("将Lambda表达式赋值给接口变量");
        obj.func();
        obj.defaultMethod();
        System.out.println(obj.hashCode());
    }
}
```

# 泛型函数式接口



在实际开发中，JDK提供了很多内置的函数式接口可供使用，这些接口大多定义为**泛型**接口。例如，以下为JDK所提供的Function接口：

```
@FunctionalInterface
public interface Function<T, R> {
    R apply(T t);
}
```

这个接口规定它所接收的Lambda表达式必须满足以下条件：

- (1) 有一个类型为T的方法参数
- (2) 返回一个类型为R的结果



要执行赋值给Function类型变量的Lambda表达式，可以通过调用它的apply()方法实现。

# 使用JDK提供的Function泛型接口示例

```
public class UseGenericFunctionInterface {  
    public static void main(String[] args) {  
        //接收的Lambda表达式满足以下要求  
        //(1)接收一个String类型的字符串参数  
        //(2)返回一个Integer类型的结果  
        Function<String, Integer> stringLength = str -> str.length();  
        //通过apply()方法调用  
        System.out.println(stringLength.apply("Hello")); // 5  
    }  
}
```

依葫芦画瓢，我们也可以定义并使用自己的函数式泛型接口.....

# 自定义泛型函数式接口

定义:

```
//自定义的接收三个参数的泛型函数式接口
@FunctionalInterface
public interface TriFunction<T, U, V, R> {
    R apply(T t, U u, V v);
}
```

使用:

```
private static void useMyGenericFunctionInterface() {
    TriFunction<Integer, Integer, Integer, Integer> add3Number =
        (x, y, z) -> x + y + z;
    //输出: 6
    System.out.println(add3Number.apply(t: 1, u: 2, v: 3));
}
```



# JDK中定义了许多的函数式接口



开发中四个非常常用的函数式接口类型：

| 名字                               | 函数签名                           |
|----------------------------------|--------------------------------|
| <code>Consumer&lt;T&gt;</code>   | <code>void accept(T t)</code>  |
| <code>Supplier&lt;T&gt;</code>   | <code>T get()</code>           |
| <code>Function&lt;T,R&gt;</code> | <code>R apply(T t)</code>      |
| <code>Predicate&lt;T&gt;</code>  | <code>boolean test(T t)</code> |

这四个函数式接口，在实际开发中用得非常多，在后面的许多例子中都会看见它们的身影.....

# 小结



函数式接口，可用于定义单独的函数式接口类型的变量，也可以用于定义另一个函数的参数类型。



函数式接口类型的变量或（方法参数），能够直接接收一个Lambda表达式。



Java 8以后，JDK中的许多原有的符合函数式接口要求的“老接口”，都添加了@FunctionalInterface注解，并且给原有接口添加了一些默认方法和静态方法，同时，JDK又添加了一些新的函数式接口。同学们必须熟悉这些接口的职责，熟练掌握其用法。